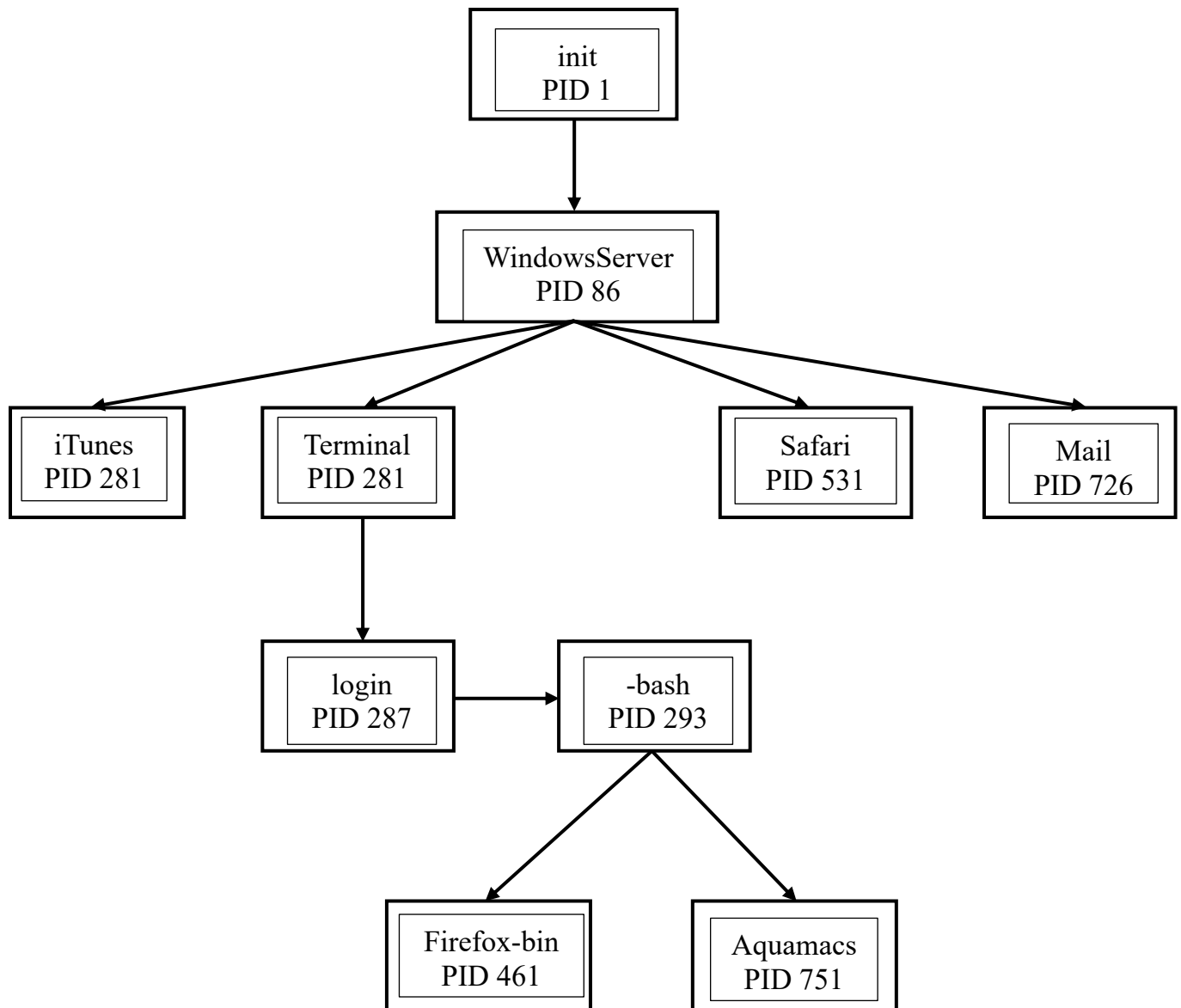


Bài 1: Quan hệ cha-con giữa các tiến trình

a. Cây parent-child



b. Dùng ps tìm tiến trình cha theo PID

ps -f -p <PID>

Cột PPID hiển thị PID của tiến trình cha. Ví dụ tìm cha của PID 287:

bash

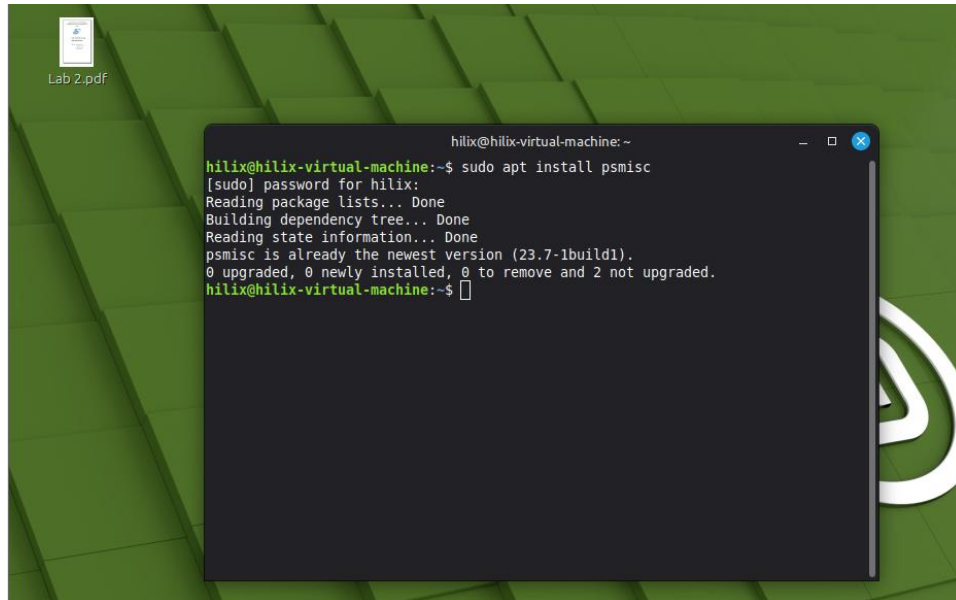
ps -f -p 287

PPID = 282 → cha là Terminal

c. Dùng pstree

1. Cài đặt pstree

sudo apt install psmisc

A screenshot of a Linux desktop environment with a green background. In the top-left corner, there is a file icon labeled 'Lab 2.pdf'. A terminal window is open in the center, displaying the command 'sudo apt install psmisc' and its output. The output shows that psmisc is already installed at the latest version (23.7-1build1) and no action is required. The terminal window title is 'hilix@hilix-virtual-machine: ~'.

```
hilix@hilix-virtual-machine:~$ sudo apt install psmisc
[sudo] password for hilix:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
psmisc is already the newest version (23.7-1build1).
0 upgraded, 0 newly installed, 0 to remove and 2 not upgraded.
hilix@hilix-virtual-machine:~$
```

2. Cách sử dụng pstree

Tìm cha của một tiến trình theo PID (ví dụ PID 461):

Sử dụng lệnh:

ps-tree -p -s 461

để tìm cha của 1 tiến trình (461) theo PID:

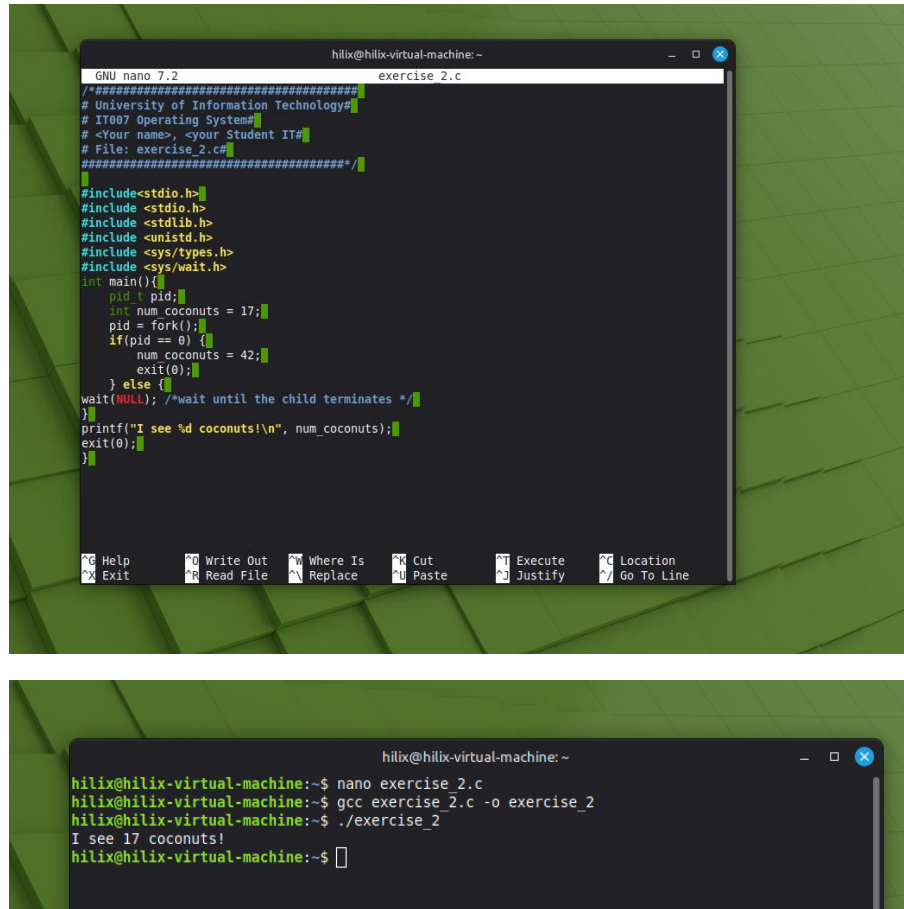
Kết quả:

```
init(1)——WindowServer(86)——Terminal(282)——login(287)——bash(293)——
firefox-bin(461)
```

Cấu trúc lệnh:

ps-tree -p -s <ID PID>

Bài 2: Chương trình in ra gì?



The first screenshot shows a nano editor window titled 'exercise 2.c' with the following code:

```
/*#####  
# University of Information Technology#  
# IT007 Operating System#  
# <Your name>, <your Student ID#  
# File: exercise_2.c#  
#####*/  
  
#include<stdio.h>  
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <sys/types.h>  
#include <sys/wait.h>  
int main(){  
    pid_t pid;  
    int num_coconuts = 17;  
    pid = fork();  
    if(pid == 0){  
        num_coconuts = 42;  
        exit(0);  
    } else {  
        wait(NULL); /*wait until the child terminates */  
    }  
    printf("I see %d coconuts!\n", num_coconuts);  
    exit(0);  
}
```

The second screenshot shows the terminal output after running the program:

```
hilix@hilix-virtual-machine: ~  
hilix@hilix-virtual-machine:~$ nano exercise_2.c  
hilix@hilix-virtual-machine:~$ gcc exercise_2.c -o exercise_2  
hilix@hilix-virtual-machine:~$ ./exercise_2  
I see 17 coconuts!  
hilix@hilix-virtual-machine:~$
```

Kết quả: I see 17 coconuts!

Giải thích:

Sau `fork()`, tiến trình con (`pid == 0`) gán `num_coconuts = 42` rồi `exit(0)`. Do tiến trình cha và con có **không gian địa chỉ riêng biệt** (bản sao độc lập), việc con thay đổi `num_coconuts` không ảnh hưởng đến cha. Cha gọi `wait(NULL)` chờ con kết thúc rồi in ra biến của chính nó — vẫn là 17.

Bài 3: Các hàm thay đổi thuộc tính pthread

Tất cả hàm liên quan đều có dạng `pthread_attr_*`. Các hàm chính:

Hàm	Công dụng
<code>pthread_attr_init(attr)</code>	Khởi tạo đối tượng attr với giá trị mặc định
<code>pthread_attr_destroy(attr)</code>	Giải phóng attr

pthread_attr_setdetachstate(attr, state)	Đặt trạng thái: PTHREAD_CREATE_JOINABLE hoặc PTHREAD_CREATE_DETACHED
pthread_attr_setstacksize(attr, size)	Đặt kích thước stack (byte)
pthread_attr_setschedpolicy(attr, policy)	Đặt chính sách lập lịch: SCHED_FIFO, SCHED_RR, SCHED_OTHER
pthread_attr_setschedparam(attr, ¶m)	Đặt độ ưu tiên (pr

Ví dụ minh họa detachstate:

```

GNU nano 7.2 exercise 3.c
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

void *thread_func(void *arg) {
    printf("Thread chạy và tự thoát (detached)\n");
    return NULL;
}

int main() {
    pthread_t tid;
    pthread_attr_t attr;

    pthread_attr_init(&attr);
    pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);
    // Thread detached: không cần pthread_join(), tài nguyên tự giải phóng khi kết
    pthread_create(&tid, &attr, thread_func, NULL);
    pthread_attr_destroy(&attr);

    sleep(1); // Chờ thread chạy xong
    printf("Main kết thúc\n");
    return 0;
}

```

```

hilix@hilix-virtual-machine: ~
hilix@hilix-virtual-machine:~$ nano exercise_3.c
hilix@hilix-virtual-machine:~$ gcc exercise_3.c -o exercise_3 -pthread
hilix@hilix-virtual-machine:~$ ./exercise_3
Thread chạy và tự thoát (detached)
Main kết thúc
hilix@hilix-virtual-machine:~$

```

Bài 4: Chương trình dùng signal

```
GNU nano 7.2 exercise 4.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t vim_pid;

void on_sigint(int sig) {
    kill(vim_pid, SIGKILL);
    waitpid(vim_pid, NULL, 0);
    printf("\nYou are pressed CTRL+C! Goodbye!\n");
    exit(0);
}

int main() {
    printf("Welcome to IT007, I am <your_Student_ID>!\n");

    vim_pid = fork();
    if (vim_pid == 0) {
        // Tách tiến trình con ra khỏi process group của cha
        setpgpr();
        execl("/usr/bin/vim", "vim", "abcd.txt", NULL);
        exit(1);
    }

    signal(SIGINT, on_sigint);
    wait(NULL);

    return 0;
}
```

[illegible]

```
hilix@hilix-virtual-machine: ~  
hilix@hilix-virtual-machine:~$ nano exercise_4.c  
hilix@hilix-virtual-machine:~$ gcc exercise_4.c -o exercise_4  
hilix@hilix-virtual-machine:~$ ./exercise_4  
Welcome to IT007, I am <your_Student_ID>!  
^C  
You are pressed CTRL+C! Goodbye!  
hilix@hilix-virtual-machine:~$
```